INFO411
Project Report

By: Sikha Nair (7810398), Sharen Thomas (7700787)
& Mishaim Jat (8792501)

TABLE OF CONTENTS

Executive Summary.....3
Explanation of Pre-processing Steps with Associate Graphs.....4
1. Data Loading and Inspection.....4
2. Categorical Features Check.....4
3. Standardization.....5
4. Dimensionality Reduction Using PCA.....6
Clustering Methods, Results and Graphs.....7
1. K-Means Clustering with Elbow Method & Silhouette Score Graphing.....7
2. Hierarchical Clustering with Dendrogram Graphing.....12
3. DBSCAN Clustering with K-Distance Graphing.....16
Discussion of Quality of Clusters.....20
1. K-Means Clustering.....20
2. Hierarchical Clustering.....21
3. DBSCAN Clustering.....21
Overall Assessment of Cluster Quality.....22
Discussion of Patterns Discovered.....23
1. K-Means Clustering Patterns.....23
2. Hierarchical Clustering Patterns.....24
3. DBSCAN Clustering Patterns.....25
Conclusion.....26
Work Distribution.....27

Executive Summary

For this project, we will be choosing to implement the US Census Data 1990. We will load the census data and drop the null values to pre-process as well as clean before we start to cluster and graph. The clusters and graphs that are created, will be used to identify any patterns, anomalies, similarities and overall give a summary of how the data is spread out.

For the pre-processing of the data, we graphed the data to give a visual representation of any null values present, so that we can easily drop those values, and the result was an empty graph which told us that the data contained no null values. In addition, the data included only numerical features, so further encoding of the code will not be performed, however, to ensure that features provided equally to the generating of clustering and graphing, standardization will be performed by the team.

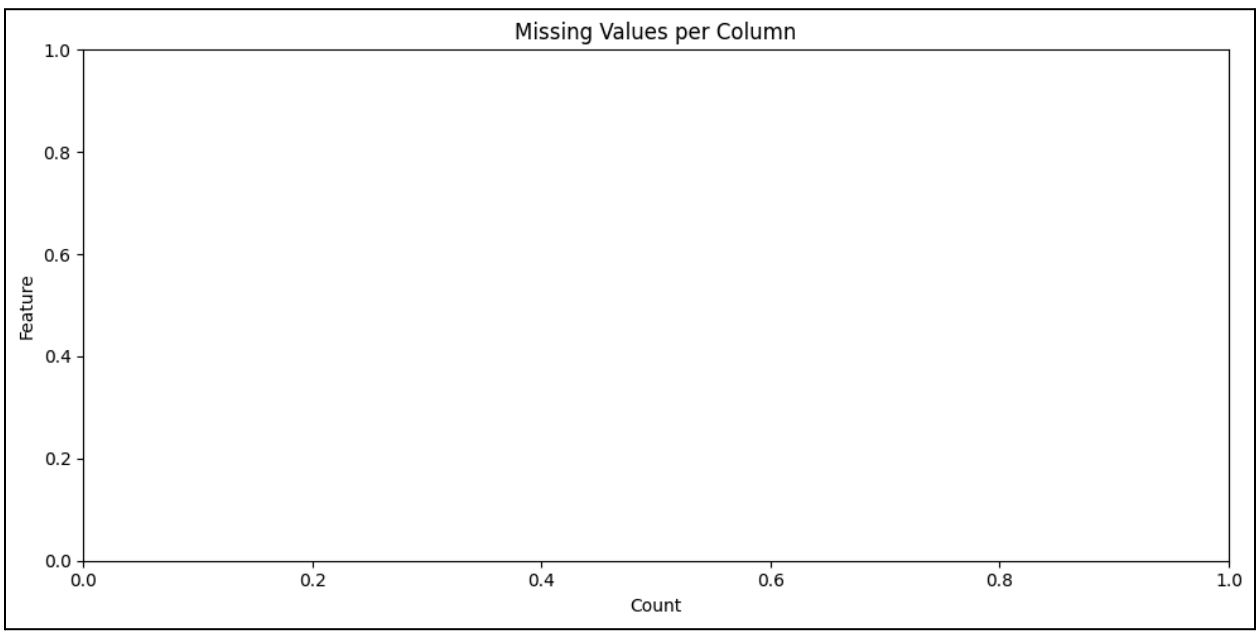
For clustering we used K-means, and density based algorithms were used. From which the elbow method, silhouette method, dendrogram and k-distance will be used to create the graphs. These methods and techniques will be used to better conclude our analysis in this project.

Explanation of Pre-processing Steps with Associate Graphs

1. Data Loading and Inspection

The dataset, **USCensus1990_data.csv**, was imported and a subset of 30,000 rows was selected for analysis. Sampling helps make the data easier and faster to work with while still keeping the main patterns and variety from the original dataset. The initial inspection focused on identifying missing values across the dataset.

- **Missing Values Check:**
  A bar plot of missing values per column was generated. However, the plot and subsequent .isnull().sum() analysis confirmed that no columns contained missing data.



2. Categorical Features Check

Using select_dtypes, we checked for categorical features that might require encoding.

```
[ ] categorical_data = data.select_dtypes(include = ['object', 'category']).columns.tolist()
    print("Categorical columns:", categorical_data)
```

```
Categorical columns: []
```

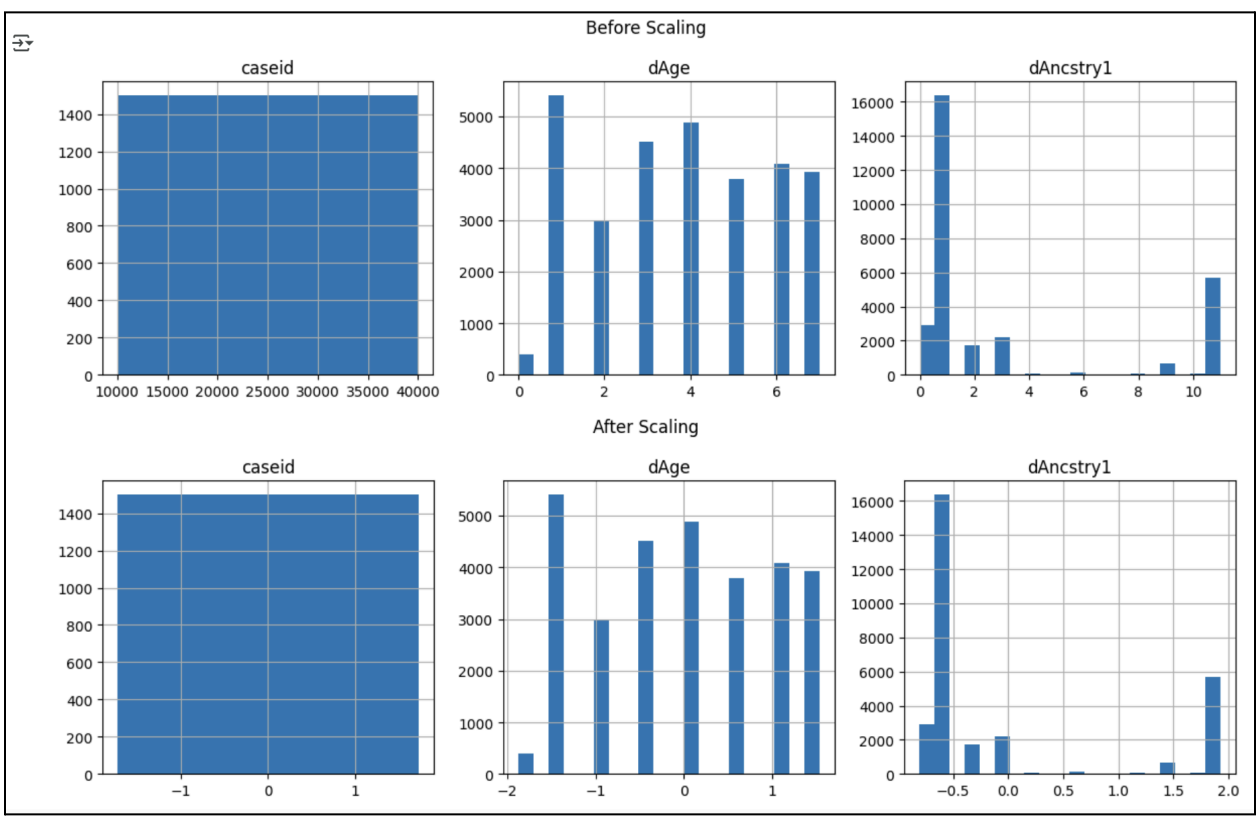
- The dataset contained no categorical columns, eliminating the need for encoding techniques like one-hot encoding.

3. Standardization

- All the data was in numbers, but the values were on very different scales (for example, case IDs had large numbers, while age had smaller ones). To make sure every feature had an equal impact—especially for methods like clustering or PCA—we used standardization.

  Why Standardize?

  - The dataset was already on a similar scale, which is why the histograms before and after scaling looked nearly the same. However, standardization was still applied to make sure all features were perfectly aligned in terms of mean and variance. This ensures fairness when using algorithms like clustering or PCA that are sensitive to feature scales—even if the original data appeared to be balanced.

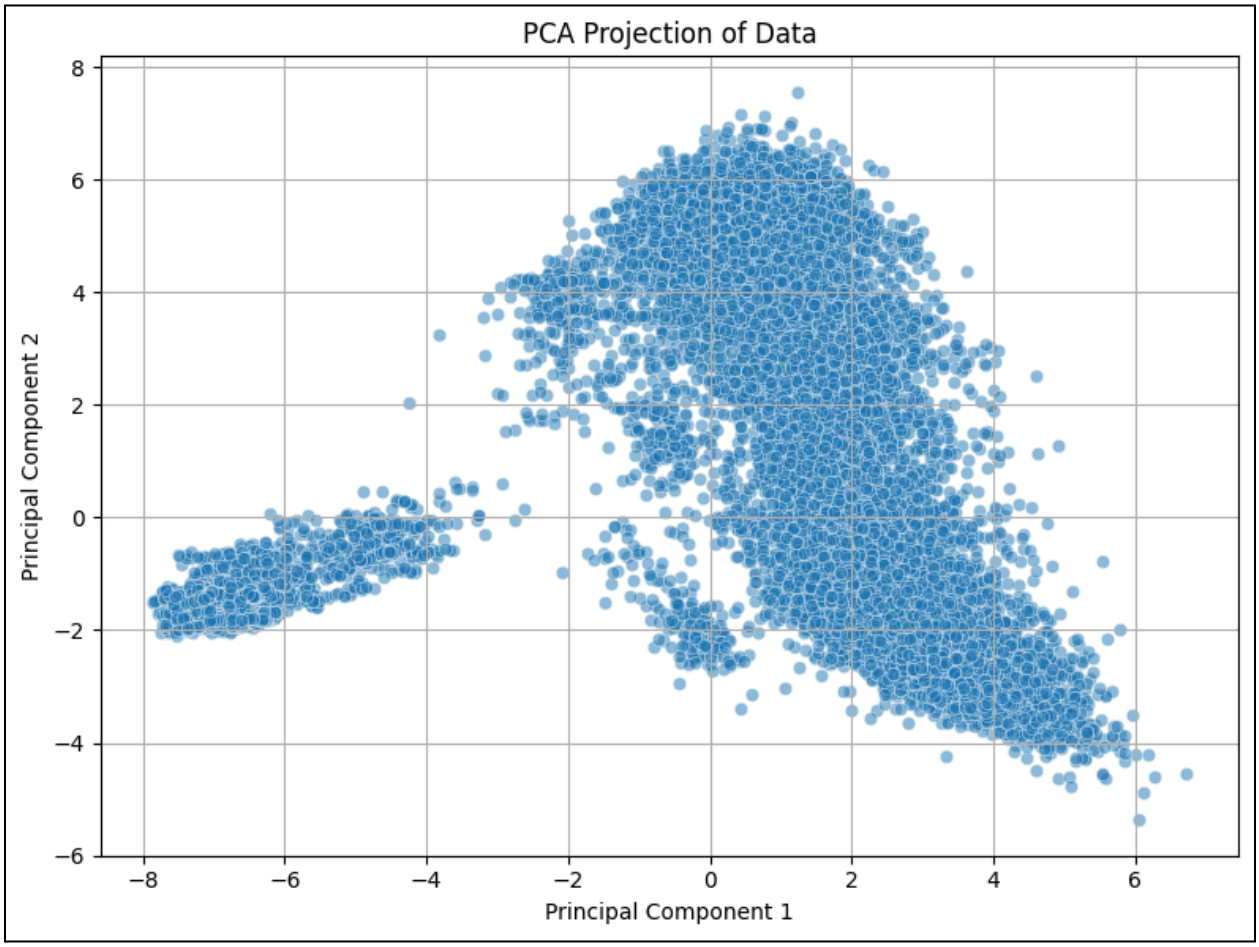


4. Dimensionality Reduction Using PCA

To better understand the data and spot patterns, we used **Principal Component Analysis** to reduce the many features down to just 2 main ones.

Then a scatter plot was generated where each dot is a data point.

Even though PCA doesn't actually group the data, the plot helps us see possible patterns or natural groupings. This can be useful later when we apply clustering methods like K-Means or DBSCAN.



Clustering Methods, Results and Graphs

1. K-Means Clustering with Elbow Method & Silhouette Score Graphing

```python
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

inertia = []
silhouette_scores = []
K = range(2, 11)

for k in K:
    kmeans = KMeans(n_clusters = k, random_state = 42)
    kmeans.fit(data_scaled)
    inertia.append(kmeans.inertia_)
    silhouette_scores.append(silhouette_score(data_scaled, kmeans.labels_))

# Plotting Elbow Curve
plt.figure(figsize = (10, 4))
plt.plot(K, inertia, marker = 'o', color = 'red')
plt.xlabel('Number Of Clusters (K)--->')
plt.ylabel('Inertia --->')
plt.title('Elbow Method For K-Means Clustering')
plt.grid(True)
plt.tight_layout()
plt.show()

# Plotting Silhouette Scores
plt.figure(figsize = (10, 4))
plt.plot(K, silhouette_scores, marker = 'o', color = 'green')
plt.xlabel('Number Of Clusters (K) --->')
plt.ylabel('Silhouette Score --->')
plt.title('Silhouette Scores For K-Means Clustering')
plt.grid(True)
plt.tight_layout()
plt.show()
```

In this section, the code helps to determine the most suitable number of clusters (k) for K-Means Clustering using:

- The Elbow Technique
- The Silhouette Score

First Step: To import the necessary libraries:

- **'KMeans':** The clustering algorithm from the 'sklearn' library.
- **'silhouette_score':** This is used to evaluate the cluster's quality.

Second Step: To initialize the lists to store the performance metrics:

- **'inertia':** This measures the cluster's compactness. Lower the better.
- **'silhouette_scores':** This measures how distinct are the clusters. Higher the better.

Third Step: To iterate through a range of k (i.e., the number of clusters):

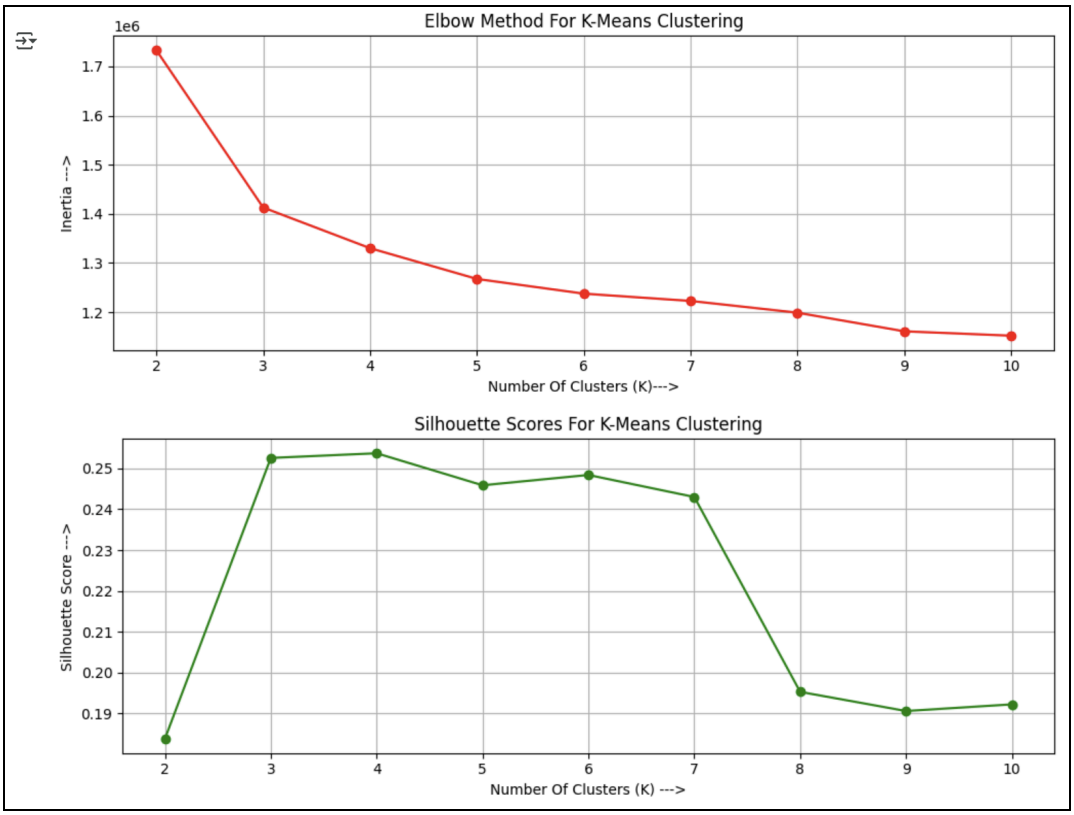
- The loop runs for k from ranges 2 to 10.
- For each k value:
  - ➔ A 'KMeans' model is instantiated with k clusters.
  - ➔ The model is trained using the 'data_scaled' dataset.
  - ➔ The inertia for the model is stored.
  - ➔ The silhouette score is computed using the cluster labels that is predicted and then added to the list.

Fourth Step: To plot the Elbow Curve (i.e., inertia versus k):

- This graph will display how the inertia varies with the different values of k.
- The "Elbow point" (i.e., the point where the line starts to bend) mostly indicates the optimal k, since adding more clusters does not decrease the inertia significantly.

Fifth Step: To plot the Silhouette Score (i.e., score versus k):

- This graph will display how the scores vary with the different values of k.
- The maximum score is usually the most suitable number of clusters, which shows that the groups are well-separated and tightly packed.



This two graphical output shows that:

- **The Elbow Method for K-Means:**
  - ➔ **X-axis** = The number of clusters (k)
  - ➔ **Y-axis** = Inertia (i.e., the total of squared distances from each point to its respective center of the cluster)
  - ➔ The curve declines rapidly from k = 2 to k = 4, and then flattens after that point
  - ➔ This is called the "Elbow point" , where increasing the number of clusters does not decrease the inertia any longer.

  Interpretation:
  - ➔ The elbow point occurs at k = 4, so this suggests choosing k = 4 is a suitable choice.
  - ➔ After this point (k = 4), the decrease in inertia is minimal, so increasing the number of clusters does not offer much benefit.

- **The Silhouette Score for K-Means:**
  - ➔ **X-axis** = The number of clusters (k)
  - ➔ **Y-axis** = The silhouette score (i.e., how distinct and clear the clusters are)
  - ➔ The peak silhouette score is achieved at k = 4 (i.e., approx 0.26)
  - ➔ Although k = 3 is close it is slightly lower than k = 4
  - ➔ Beyond this point, the score drops, especially a sharp drop at k = 8 to k = 10

    Interpretation:
    - ➔ The highest silhouette score marks at k = 4, which suggests a well-separated and compactness among the clusters.
    - ➔ High values like k = 5 to k = 10 display poor quality of clustering.

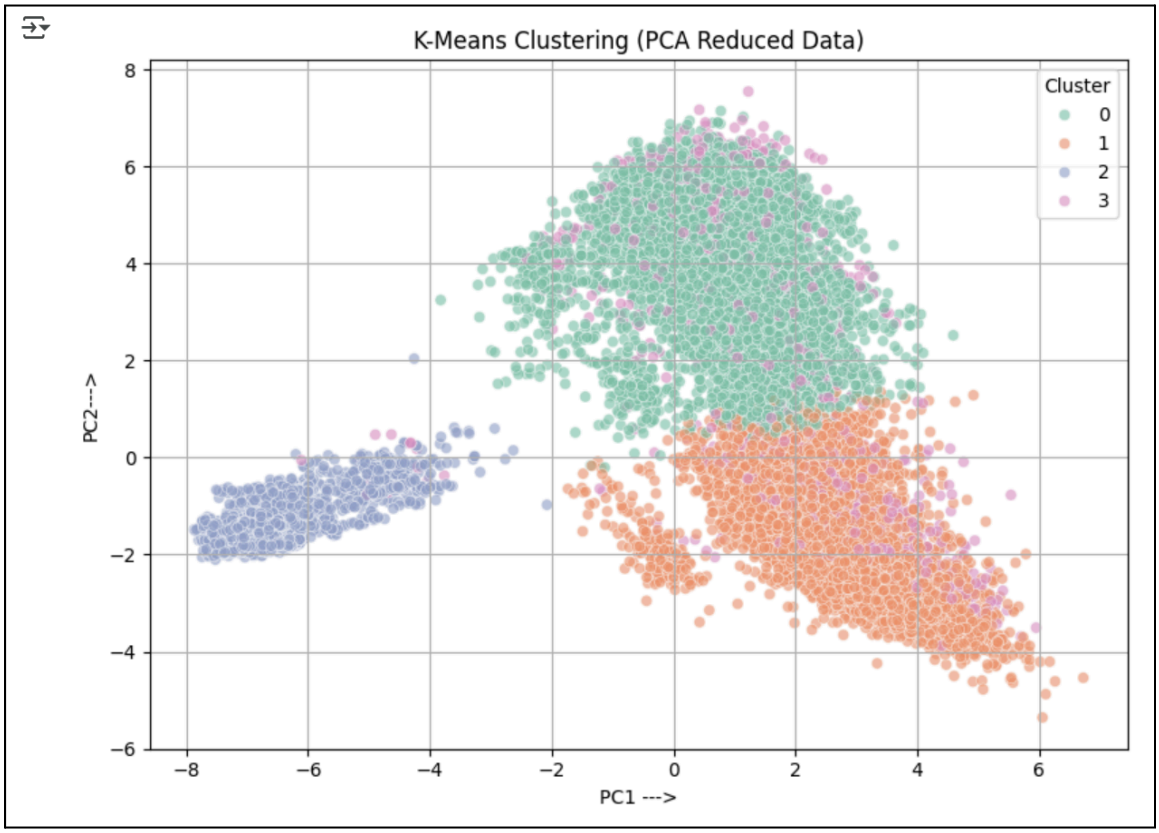
```
# Applying K-Means with optimal_k according to Elbow/Silhouette
optimal_k = 4  # This is adjusted after observing the Elbow plot above^
kmeans = KMeans(n_clusters = optimal_k, random_state = 42)
labels_kmeans = kmeans.fit_predict(data_scaled)

# Visualizing the Clusters
plt.figure(figsize = (8, 6))
sns.scatterplot(x = data_pca[:, 0], y = data_pca[:, 1], hue = labels_kmeans, palette = "Set2", alpha = 0.6)
plt.title("K-Means Clustering (PCA Reduced Data)")
plt.xlabel("PC1 --->")
plt.ylabel("PC2--->")
plt.legend(title="Cluster")
plt.grid(True)
plt.tight_layout()
plt.show()
```

In this section, based on the previous evaluation, k = 4 is chosen as the most suitable number. This value is manually set based on the earlier plots.

- **'KMeans':** Is initialized with the n_clusters = 4
- The model is trained on the normalized data (data_scaled) and then predicts each data point to a cluster.
- **'labels_kmeans' :** It contains the number of clusters (From 0 to 3) that is corresponding to each data point.

- For the scatter plot, it uses Seaborn:
  - ➔ **'data_pca[:, 0]'** and **'data_pca[:, 1]'** represent the top two principal components (which are extracted through PCA).
  - ➔ **'hue = labels_kmeans' :** Based on the cluster labels, it colors the data points
  - ➔ **'palette = "Set2"' :** provides different soft colors.

➔ **'alpha = 0.6' :** Sets the points partially transparent.
➔ **'plt.grid(True)':** Iis added with auto-adjusts layout to prevent overlapping.



This graphical output shows that:

- It indicates that K-Means was applied for clustering the data, and the dataset was reduced to two dimensions through Principal Component analysis (PCA) for a better visualization.
- The data points are colored according to the cluster it belongs to, that is, there are 4 clusters present, where each are represented with different colors like blue, green, orange and pink shades.

Interpretation:

- **Cluster Separation:**
  ➔ The blue shade cluster (Cluster - 2) is clearly isolated, which implies it is distinct in the dataset.

- ➔ The green shade cluster (Cluster - 0) and orange shade cluster (Cluster 1) exhibit some overlap, which indicates that their underlying features may be similar in certain dimensions.
- ➔ The pink cluster (Cluster 3) appears to be scattered among the other groups, which suggests it might be smaller or indistinct grouping that potentially consists of outliers or noise.

- **Cluster Density:**
  - ➔ The green shade cluster (Cluster - 0) and orange shade cluster (Cluster - 1) are larger and denser, which indicates that they contain most of the data points.
  - ➔ The blue shade cluster (Cluster - 2) is smaller in size but are closely packed, which implies it forms a compact group.
  - ➔ The pink cluster (Cluster - 3) appears to be dispersed and sparse, which suggests it might not be well-defined such as the mixed points or outliers.

2. Hierarchical Clustering with Dendrogram Graphing

```
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster

# Using linkage on PCA-reduced data
linked = linkage(data_pca, method = 'ward')

# Plotting dendrogram
plt.figure(figsize = (12, 6))
dendrogram(linked, truncate_mode = 'lastp', p = 30, leaf_rotation = 90., leaf_font_size = 10., show_contracted = True)
plt.title("Dendrogram For Hierarchical Clustering")
plt.xlabel("Cluster Size --->")
plt.ylabel("Distance --->")
plt.tight_layout()
plt.show()
```

In this section, the code does the following:

First Step: To import the necessary functions:

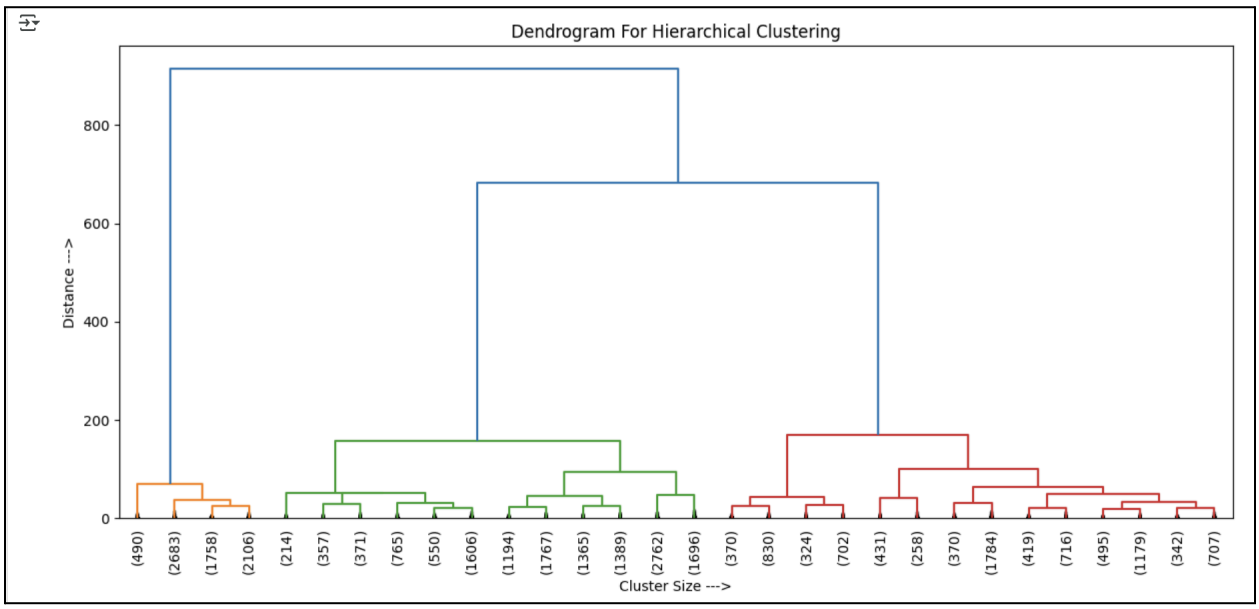
- **'linkage':** This is to compute the clustering
- **'dendrogram':** This will plot the hierarchy
- **'fcluster':** This is not exactly used here in the code but from the hierarchy, it assigns the clusters.

Second Step: To perform the linkage on reduced dataset of PCA

- It performs the agglomerative clustering through the **‘ward’** method. This parameter ensures that intra-cluster variance is reduced when merging, a common method for numerical data.
- **‘data_pca’** is the dataset that has been reduced to fewer dimensions via PCA.

Third Step: To plot the dendrogram

- Generates a dendrogram to show how the clusters are merged.
- **‘truncate_mode = 'lastp''**: It displays only the final 30 merges (Avoids clusters).
- **‘p = 30’**: Displays the final 30 merged clusters.
- **‘leaf_rotation = 90’**: For readability, the leaf label is rotated.
- **‘leaf_font_size=10’**: Font size is set for the leaf labels.
- **‘show_contracted = True’**: This shows the summary for the clusters that are not expanded.
- **‘plt.tight_layout()’**: This is to prevent overlapping of text



This graphical output shows that:

- Every leaf (The bottom label) stands for a data point or a cluster (The number in brackets indicates the size of the cluster).
- The Y-axis displays how separate the clusters were before they were merged.
- The merge from the base to the top displays how small clusters or the individual points combine into the larger ones.

- The three main color segments are orange, green and red shades, which suggests a natural grouping into three prominent clusters based on the cut level.
- The tall jumps in height such as above 600 between the merges, signifies large dissimilarity, ideal to choose the cluster count.

Interpretation:

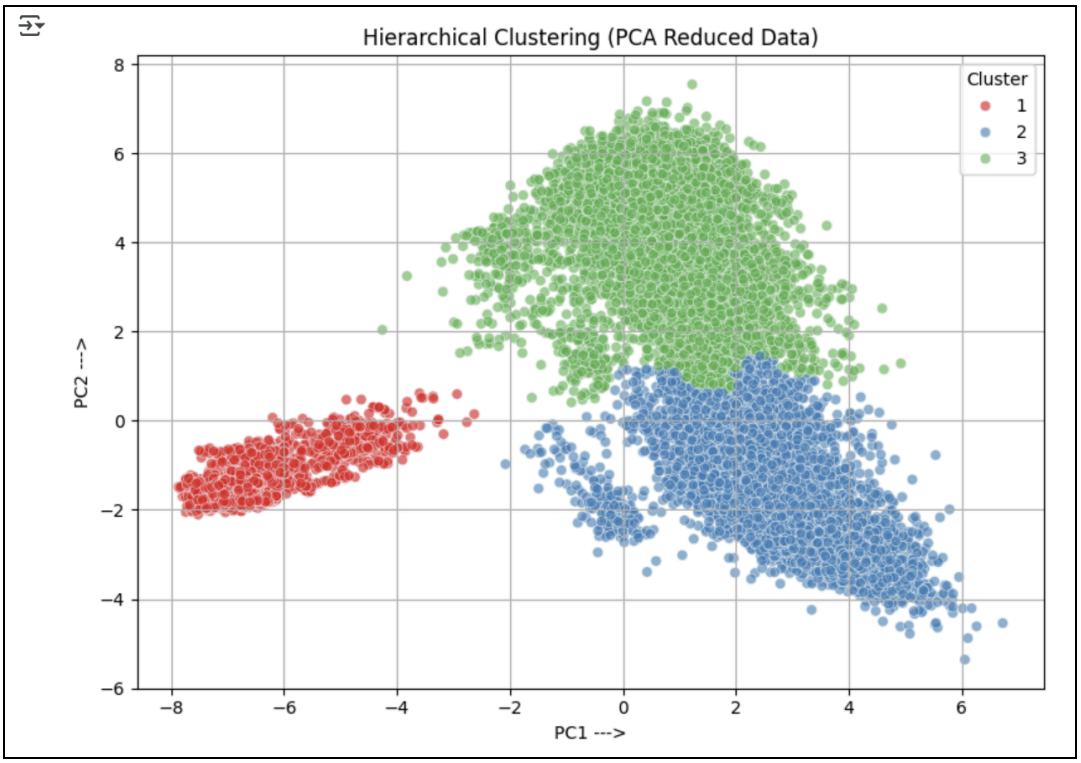
➔ It's possible to cut the dendrogram right across the large height jump (~600 - 900).

➔ This result indicates an optimal cluster count of 3.

```
# Based on the dendrogram, the clusters are chosen as above^
num_clusters = 3
labels_hier = fcluster(linked, num_clusters, criterion = 'maxclust')

# Visualizing the Clusters
plt.figure(figsize = (8, 6))
sns.scatterplot(x = data_pca[:, 0], y = data_pca[:, 1], hue = labels_hier, palette = "Set1", alpha = 0.6)
plt.title("Hierarchical Clustering (PCA Reduced Data)")
plt.xlabel("PC1 --->")
plt.ylabel("PC2 --->")
plt.legend(title="Cluster")
plt.grid(True)
plt.tight_layout()
plt.show()
```

In this section, based on the previous evaluation:

- **‘num_clusters = 3’:** This defines the number of clusters to be extracted from the dendrogram by observing the structure.
- **‘fcluster(linked, num_clusters, criterion='maxclust')’:** This applies a linkage matrix to categorize every data point to a cluster, using ‘criterion='maxclust'’ as a constraint it tells to form exactly three.
- **‘labels_hier’:** This holds the cluster labels that are assigned to every data point, reflecting the hierarchical clustering.
- This mainly helps us to know how to evaluate visually on how well it separated the data into clusters.



This graphical output shows that:

- This plot shows that the data is classified as 3 clusters, as it is specified by 'num_cluster = 3'.
- The plot utilizes PCA for reduction of high-dimensional data to 2Ds, which is PC1 (x-axis) and PC2 (Y-axis) for a better visualization.
- Every point corresponds to a data item, and the colors (Red, green and blue) indicate which cluster (Cluster 1, 2 and 3) each belongs to.

Interpretation:

➔ The clusters are visually distinct, particularly Cluster 1 (Red), which is positioned far from the other clusters. Cluster 2 (Blue) and Cluster 3 (Green) slightly overlap but they can still be distinguished.

➔ Cluster 1 is tight and dense, which indicates the data points are closely related, whereas Cluster 2 and 3 are more spreaded out, which indicates there are more variations among those groups.

➔ This plot validates that hierarchical clustering was a success in categorizing the data into meaningful clusters.

3. DBSCAN Clustering with K-Distance Graphing

```
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN
import numpy as np

# Plotting of K-Distance to find the Optimal Eps
neighbors = NearestNeighbors(n_neighbors = 5)
neighbors_fit = neighbors.fit(data_scaled)
distances, indices = neighbors_fit.kneighbors(data_scaled)

# Sorting the distances and then plotting
distances = np.sort(distances[:, 4])  # 5th neighbor (Index 4)
plt.figure(figsize = (10, 5))
plt.plot(distances)
plt.axhline(y = 3.0, color = 'red', linestyle = '--')  # Adjusted the threshold based on Elbow
plt.title("K-Distance For DBSCAN Clustering")
plt.xlabel("Data Points sorted by distance --->")
plt.ylabel("5-NN Distance --->")
plt.grid(True)
plt.tight_layout()
plt.show()
```

In this section, the code does the following:

First Step: To import the necessary libraries:

- **'NearestNeighbors'** from **'sklearn.neighbors'** is utilized to compute the distance to the neighbours who are the nearest.
- **'DBSCAN'** is utilized later for clusters based on density.
- **'numpy'** for the numerical computations.

Second Step: To initialize NearestNeighbors:

- This builds a model to calculate the five closest points for every data point.
- **'n_neighbors = 5'** implies we are focusing at the distance to the fifth closest point, which is a standard approach in DBSCAN elbow analysis.

Third Step: To fit the model:

- The model is trained on a **'data_scaled'**.
- It calculates the distances from every data point to its five closest neighbors.
- **'distances'** array is where every row of values contains distances to the five closest neighbors for a data point.

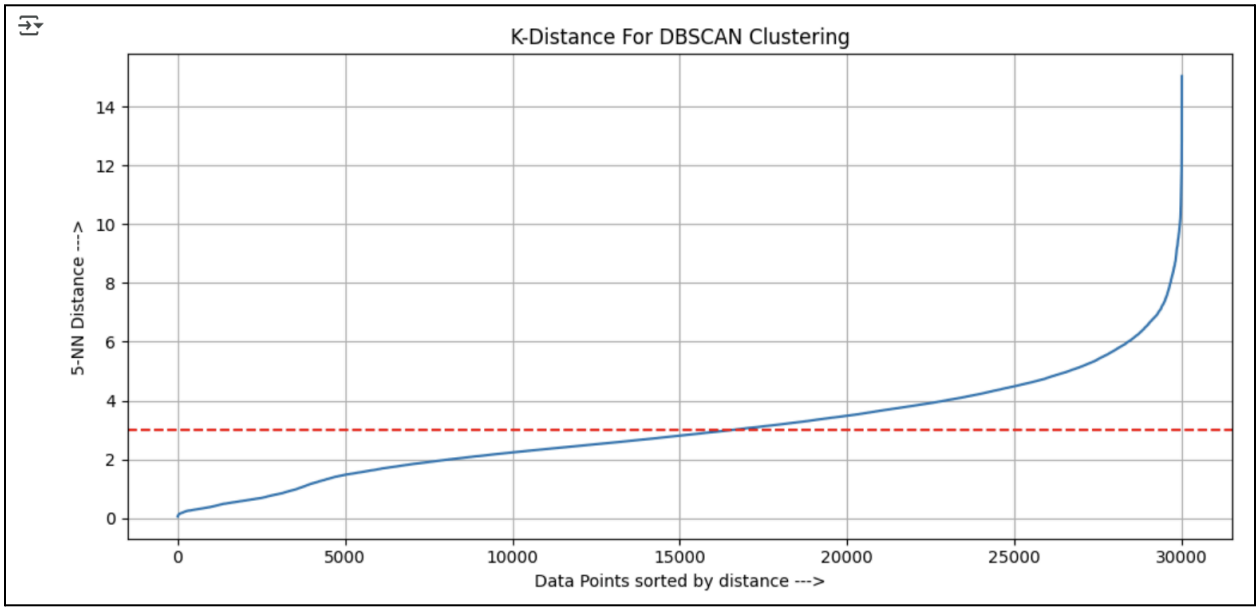
Fourth Step: To sort the distances for the fifth neighbor:

- For every point, choose the fifth closest neighbour's distance (Index 4).

- Organizing these distances enables us to find the ‘knee’ (Elbow point) in the curve that is used to estimate the ‘eps’ value.

Fifth Step: To plot the K-Distance graph:

- This displays the sorted distances to observe where a steep escalation takes place.
- This plot reveals the transition from dense regions to sparse regions.
- A horizontal line is plotted at ‘y = 3.0’ to indicate a threshold, which serves as the eps value.
- This is essential in DBSCAN to differentiate core points from the noise.



This graphical output shows that:

- **X-axis** = This shows the data points that are sorted by distance to their fifth closest neighbor.
- **Y-axis** = This displays the distance to the fifth closest neighbor for every point.

Interpretation:

➔ The curve increases gradually in the beginning (Which indicates the points in the compact clusters).

➔ The sharp upward curve is the ‘Elbow’ or the ‘Knee point’.

➔ Beyond the elbow, the distances rise rapidly (Indicating the noisy or sparse data).

➔ Highlights the selected eps (Epsilon) value.

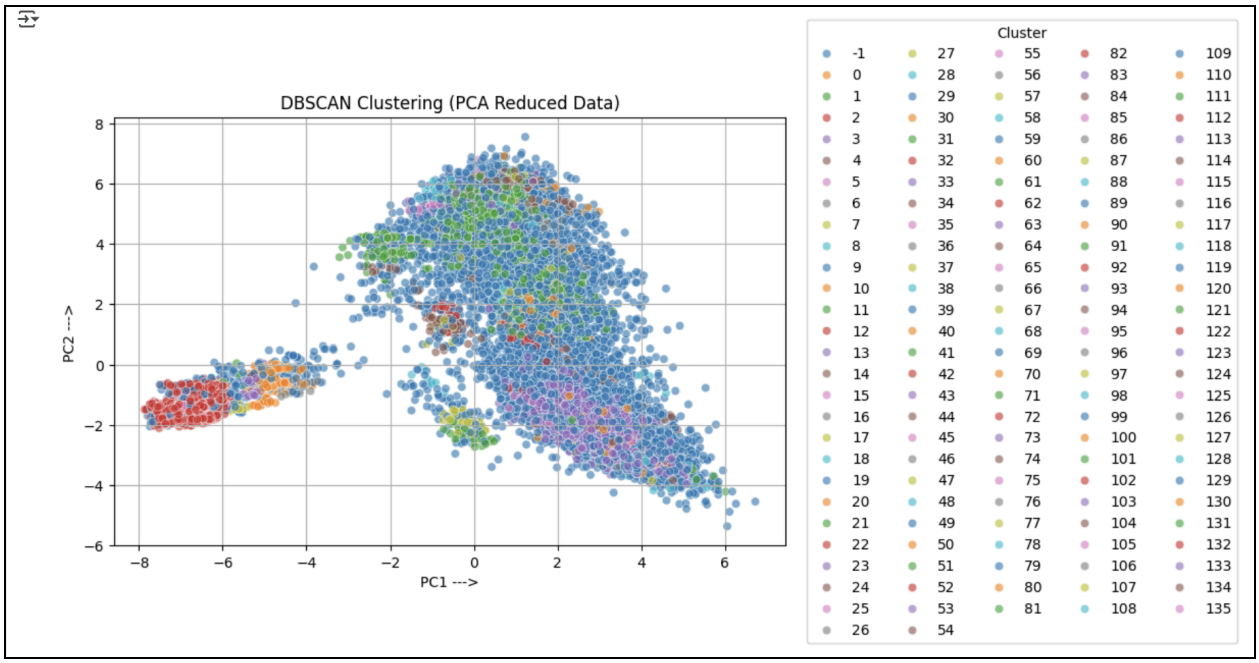
➔ This threshold value enables DBSCAN to determine whether the point lies in the dense region (Core) or not noise/border.

➔ The elbow is where it's the end of dense clusters and noise begins.

```
# Applying DBSCAN
eps_value = 3.0  # Based on Elbow of K-Distance graph
dbscan = DBSCAN(eps = eps_value, min_samples = 5)
labels_dbscan = dbscan.fit_predict(data_scaled)

# Visualizing the Clusters
plt.figure(figsize = (14, 6))  # Made it wider to accommodate the legend without overlapping the graph
sns.scatterplot(x = data_pca[:, 0], y = data_pca[:, 1], hue = labels_dbscan, palette = "tab10", alpha = 0.6)
plt.title("DBSCAN Clustering (PCA Reduced Data)")
plt.xlabel("PC1 --->")
plt.ylabel("PC2 --->")

# Legend with 5 rows (5 entries per row = multiple columns)
plt.legend(title = "Cluster", loc = 'center left', bbox_to_anchor = (1.02, 0.5), ncol = 5,frameon = True)
                                                                    # 5 columns = 5 labels per row,

plt.grid(True)
plt.tight_layout(rect = [0, 0, 0.85, 1])  # Reserving space on the right side
plt.show()
```

In this section, based on the previous evaluation,

- The value '3.0' is selected using the elbow point examined in the K-Distance graph.
- **'min_samples = 5':** Is the minimum points to create a dense region (Core point).
- **'-1':** Refers to the point as noise (Not in any cluster).
- **'palette = "tab10"':** Gives 10 different colors.
- **'plt.legend':** This keeps the legend outside the graph on the ride side.
- **'ncol = 5':** Means the cluster labels are in 5 columns.
- **'tight_layout(...)':** This is to reserve some space on the right for the legend so that it does not overlap the plot.



This graphical output shows that:

- Each data point represents a single observation from the dataset.
- They are transformed into 2D space utilizing the PCA for visualizing.
- More than 130 clusters were identified, suggesting that the dataset has many dense groups.

Interpretation:

➔ The different colors display different clusters that are detected by DNSCAN.

➔ The blue colored points with the label -1 are considered as outliers or noise, these data points do not belong to any cluster group according to the criteria of DBSCAN's.

➔ Tightly packed regions with just one color represent valid high-density groupings.

➔ DBSCAN clusters are based on the density and not based on shapes or distance.

Discussion of Quality of Clusters

1. K-Means Clustering

We performed K-Means Clustering on the standardized dataset, and evaluated it with different values of K ranging from 2 to 10.

- **Elbow Method**
  The Elbow Curve clearly revealed a strong bend at K = 4, where the inertia sharply decreased before stabilizing. This suggests that adding the number of clusters more than 4 does not significantly enhance the cluster compactness.
  - ➔ Choosing K = 4 is an ideal choice because it balances between model's interpretability and complexity, while minimizing the risk of overfitting.

- **Silhouette Score**
  Although the Elbow Method is sufficient to assess clustering, incorporating Silhouette Score delivers a quality check, deeper insights and strengthens the credibility of the results. The Silhouette Score reached its maximum at K = 4, suggesting high cohesion within the clusters and clearly separated from one another. This score confirmed the Elbow Method's result.
  - ➔ A higher Silhouette Score indicates that the data points are more closely arranged to their own cluster's center than to the other clusters.

- **Visual Confirmation**
  The PCA visualization displayed that the clusters created by K-Means were clearly separated with little overlap, particularly considering the dimensionality reduction.
  - ➔ **Overall quality:** K-means showed excellent results, as it is well-suited for the spherical nature of data, which matched its assumptions well.

2. Hierarchical Clustering

We performed Agglomerative Hierarchical Clustering with Ward's method of linkage, which focused on reducing the variance within each of the clusters.

- **Dendrogram Analysis**
  The Dendrogram structure showed a natural cutoff point at clusters of 3, marked by a long vertical gap between the successive linkages.
  - ➔ This implies that the dataset has a nested pattern, and with three major, separate clusters exist.

- **Visual Confirmation**
  The clusters displayed distinct and clear vertical or horizontal patterns, but had slightly more overlap and less precise boundary definitions than K-Means Clustering, which is expected as Hierarchical Clustering does not optimize centroids.
  - ➔ **Overall quality:** Hierarchical showed acceptable to good results, as it is suitable for detecting nested relationships or in cases where it is not necessary to define the number of clusters in advance.

3. DBSCAN Clustering

We performed DBSCAN Clustering which is a density-oriented method that is capable of identifying clusters that are non-spherical and spotting outliers, distinguishing it from the other two clustering methods.

- **K-Distance Plot**
  By analysing the 5-NN Distance Curve, we identified a noticeable bend (Elbow) at Eps (Epsilon) = 3.0, indicating it is the best choice for neighborhood radius parameter.
  - ➔ We set MinPts = 5, according to the standard guidelines for moderate size of datasets.
  - ➔ DBSCAN successfully detected the core clusters and classified outliers as noise (i.e., label = -1). It also discovered clusters with various shapes, and density, that are often difficult for K-Means Clustering to capture.

- **Visual Confirmation**
  The PCA visualization displayed that DBSCAN effectively captured most of the dense clusters and distinctly isolated the outliers. However, it was quite sensitive to the value of epsilon, and a minor change might result in significantly different clusterings.
  - ➔ **Overall quality:** DBSCAN showed reasonably good to excellent results as it was well-suited for identifying unusual shapes of clusters and outliers, but it requires precise tuning of parameters.

Overall Assessment of Cluster Quality

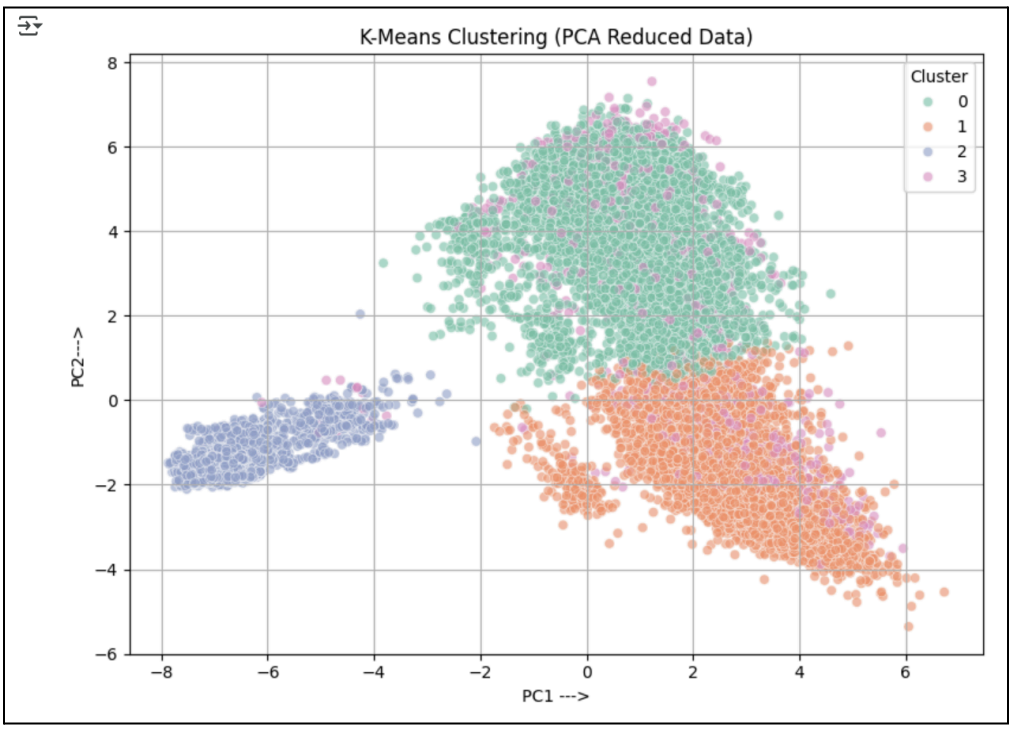
Clustering Method	Cluster Quality
K-Means	● High separation and high cohesion ● Best suited for spherical clusters ● Vulnerable to outliers ● Requires the number of clusters beforehand.
Hierarchical	● Detects nested patterns ● Offers adjustable count of clusters ● Manages clusters of non-spherical better ● Vulnerable to noisy data
DBSCAN	● Identifies outliers clearly ● Discovers clusters that are irregularly shaped ● Cluster count id not required ● Parameter tuning heavily influences the results

- K-Means Clustering offers well-defined and easily interpretable clusters, showing tight groupings and separated boundaries.
- Hierarchical Clustering offers layered structures within the data and gives a more flexible interpretation.
- DBSCAN Clustering offers to spot outliers and identify clusters that are oddly shaped that K-Means often miss to detect.

Discussion of Patterns Discovered

When looking at the results from K-Means, Hierarchical, and DBSCAN clustering, we found some clear and meaningful patterns in the data. These patterns help us understand how the data is grouped, how different the groups are from each other, and what kind of structure the data has overall. This gives us a better idea of what's going on in the dataset and points us in helpful directions for deeper analysis.

1. K-Means Clustering Patterns



Through the application of K-Means with **k = 4**, the following patterns were identified:

- **Distinct Cluster Formation**:
  The data was naturally grouped into four well-separated clusters, each reflecting a different subset of the population. The **blue cluster (Cluster 2)** stood out as particularly distinct, possibly indicating a unique behavioural or demographic segment.

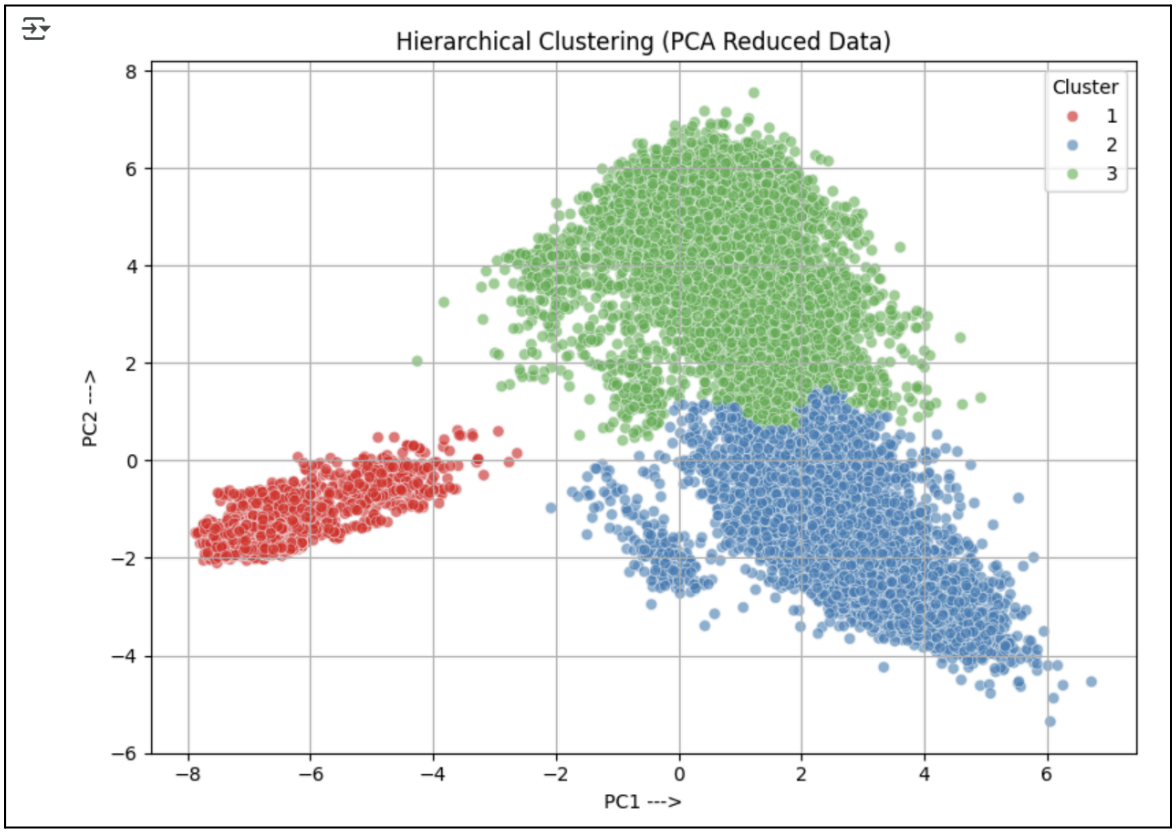
- **Overlap and Similarity**:
  The green (Cluster 0) and **orange (Cluster 1)** clusters had some overlap, which

suggests that the data points in these groups might share similar characteristics or be close to each other in certain ways. This could mean there's a mix or gradual shift between the two groups, rather than a sharp and distinct boundary.

- **Presence of a Sparse Group**:
  The **pink cluster (Cluster 3)** appeared less dense and more dispersed, which may represent outliers, mixed categories, or individuals that do not strongly align with any of the primary groupings.

- **Cluster Density Patterns**:
  Cluster 0 and Cluster 1 contained the bulk of the dataset and were more compact, hinting at a central grouping of data with shared characteristics. Cluster 2, although smaller, was tightly packed—indicative of a focused and consistent group.

2. Hierarchical Clustering Patterns



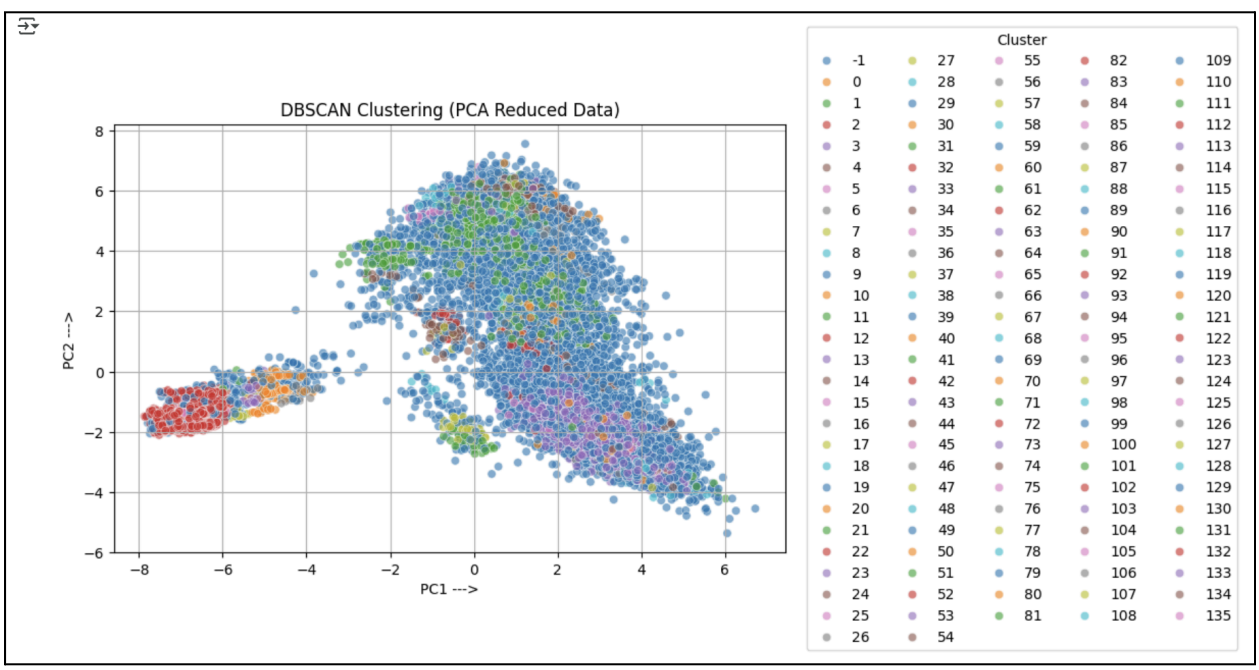
Hierarchical clustering revealed a nested structure with **3 prominent groupings**, providing a higher-level view of how data points aggregate:

- **Macro-Level Groupings**:
  Looking at the dendrogram and cutting it around the 600–900 mark, three large clusters appeared. These matched up somewhat with the K-Means results, but they gave a more general idea of how the data is structured.

- **Distinctive Outer Group**:
  One of the clusters **(Cluster 1, Red)** stood out as highly separate from the rest, implying a unique group that could represent an edge case, specific category, or isolated behavior.

- Partial Overlaps and Nested Structures:
  The slight blending between **Cluster 2 (Blue) and Cluster 3 (Green)** indicates these clusters might be subgroups within a broader category, potentially pointing to hierarchical relationships within the data.

3. DBSCAN Clustering Patterns



DBSCAN gave us a deeper look at the data, showing patterns that other methods might miss:

- **High-Granularity Clustering**:
  More than 130 clusters were detected, which suggests that the data contains many small, dense regions. This could represent micro-segments or very specific behavior clusters.

- **Outlier Detection**:
  A large number of points were labeled as **noise (label = -1)**, indicating that these points do not belong to any dominant group. These outliers could be anomalies, rare behaviors, or errors.

- **Irregular Cluster Shapes**:
  Unlike K-Means or Hierarchical, DBSCAN identified clusters of non-spherical shapes, capturing natural groupings in the data that traditional distance-based models might fail to detect.

- **Density-Based Grouping**:
  It grouped points based on how tightly they're packed together, not on how close they are to a center. This helped reveal natural clusters that other methods might overlook.

Conclusion

In conclusion, this project has widened how we can handle and analyse vast amounts of data. We were able to clearly eliminate the null values to get a clean set of data, from which we did three types of clustering such K-Means, Hierarchy and Density Based to better see the data in a visual manner. With the preprocessed data we also graphed the data to connect the clustering data to conclude what we understood from the whole experience. We used 4 types of graphs that fall under clustering methods to analyse it better such as elbow method, silhouette method, dendrogram and k-distance. In addition, this enabled us as a team to learn and understand how to standardize, clean and create visual clusters and graphs with a great amount of data.

Work Distribution

Done By (Name)	Done By (Student_ID)	Parts
Sikha Nair	7810398	● Executive Summary ● Conclusion ● Dataset extraction & creation
Sharen Thomas	7700787	● Coding part for Clustering & Graphing ● Clustering Methods, Results & Graphs ● Discussion of Quality of Clusters
Mishaim Jat	8792501	● Coding part for Pre-processing & Standardization ● Explanation for Pre-processing steps with Graphs ● Discussion of Patterns Discovered